

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

K. Vishal Reddy

Department of Computer Science
& Engineering
Keshav Memorial Institute of
Technology
Hyderabad, Telangana, India
kasarlavishalreddy@gmail.com

Anisha Paturi

Department of Computer Science
& Engineering
Keshav Memorial Institute of
Technology
Hyderabad, Telangana, India
paturi.anisha@gmail.com

Parinamika Bhanu Ch

Department of Computer Science
& Engineering
Keshav Memorial Institute of
Technology
Hyderabad, Telangana, India
chparinamikabhenu@gmail.com

Venkata Vahini Ch

Department of Computer Science
& Engineering
Keshav Memorial Institute of
Technology
Hyderabad, Telangana, India
vahini.venkata02@gmail.com

Sravani Janak

Department of Computer Science
& Engineering
Keshav Memorial Institute of
Technology
Hyderabad, Telangana, India
sravanijanaka@gmail.com

Abstract---Cloud Infrastructure-as-Code (IaC) has bugs that make systems unsafe for production. Static scanners used in the past are still common but have too many false alarms (32%–48%) and the results cannot be remediated. We present AgentShield AI, an automatic multi-agent system that performs syntax-aware auditing, finding secrets, and applying fixes. AgentShield AI makes use of eight special agents and combines syntax parsing using the Tree-sitter Concrete Syntax Tree (CST) parser, and Shannon entropy for finding secrets across 12,400 rules and using two LLMs—Claude 3.5 Sonnet and GPT-4o. Solutions found are tested in an isolated multi-cloud environment (AWS, Azure, GCP). When tested on 2,450 IaC samples, AgentShield AI was found to have 99.1% precision, 98.4% recall, and 98.7% F1 score, which is highly significant ($p < 0.001$). The method provides 97.8% successful results after the first run with 1.84 seconds response time, which inspired 94.2% reduction in the time that devs need to fix their systems.

Index Terms---Infrastructure-as-Code (IaC) Security, Multi-Agent Systems, Concrete Syntax Trees, Secret Interception, Shannon Entropy, Retrieval-Augmented Generation, Multi-Cloud Sandbox, Automated Vulnerability Remediation.

I. Introduction

Provisioning of cloud infrastructure has been undergoing a significant transformation towards a declarative approach with the advent of Infrastructure-as-Code (IaC) solutions [1]. Complex multi-cloud designs spanning Virtual Private Clouds to distributed storage solutions and clustering frameworks and across multiple distinct identity networks now require very specific programming languages such as HCL for Terraform to CloudFormation manifest for YAML/JSON/Kubernetes [2], [3]. Fully automatic pipelines from Continuous Integration through Continuous Deployment are able to interpret these textual statements to provision or update enormous configurations of cloud resources in minutes, eliminating the problem of misconfigured cloud resources and providing a reliable state-of-the-art reproducible infrastructure as first-class middleware [4].

The high-risk threat environment of software-defined infrastructure: Since the IaC templates are the actual documents that model the architecture to be applied, each security error in the template's frames directly translates into a security error in the comparable production system [5].

Stochastic Hallucination in Generative Models: While LLMs are highly competent at generating software code, uncurated generative models produce completely fictitious resource characteristics or faulty provider code, resulted in resource dependency violations and up to 28.8% errors in runtime deployment [12], [13].

4. High-Entropy Token Collisions in Secret Detection: Signature-based scanners do not detect any obfuscated passwords, and untrained Shannon entropy models generate a vast number of false positives for pseudo-random and structured strings such as hashes of encryption and UUIDs [14].

AgentShield AI Architecture and Key Contributions: To address these challenges, we present AgentShield AI, a self-governing, event-driven multi-agent system that performs the complete syntactic audit, secret interception, consensus-based remedial actions, and validated patch formation through a multi-cloud sandbox. Our major contributions are:

- The distinct decentralized 8-agent architecture using typed state contracts (Gamma) orchestrated by an asynchronous event manager.
- The innovative Tree-sitter Concrete Syntax Tree (CST) technique to solve dynamic expressions and filter out unnecessary graph paths.
- The integrated two-in-one secret interception technique that merges 140+ regular expression signatures with sliding Shannon entropy ($H \geq 4.5$) and CST stopword filtering.
- The novel hybrid dense-sparse RAG technique that blends Qdrant HNSW embeddings with BM25 lexical ranking through the Reciprocal Rank Fusion (RRF) technique using a set of 12,400 different CIS/NIST rules.
- The collaborative dual-LLM consensus mechanism (Claude 3.5 Sonnet + GPT-4o) coupled with a dual-assessment two-layer multi-cloud sandbox (LocalStack, Azurite, GCP emulators).

II. Related Work

A. Static Analysis and Pattern-Based Linters: Checkov [6] was one of the earlier tools in this space. It parses configuration files into Python AST structures and checks them against CIS benchmark rules. tfsec [7] does something similar but is built specifically for Terraform HCL — it loads the code into Go memory structures and looks for anti-patterns. KICS [8] takes a slightly different path: it converts multi-format IaC files into normalized JSON and runs them through Open Policy Agent (OPA) Rego rules. Trivy [9] is a bit different from the other three since it wasn't designed only for IaC — it also scans container images and Kubernetes manifests alongside Terraform files.

The problem with all of these tools is that they rely on pattern matching, and pattern matching does not understand how variables move across modules. Ternary expressions, dynamically built resource names, cross-module variable references — none of that gets resolved properly, so these scanners end up with false-positive rates between 32.4% and 47.9% [10]. That is almost half of what gets flagged being wrong, which is a lot for a security team to deal with every day. On top of that, none of these tools actually fix anything. They just report the problem and leave it there.

B. Policy-as-Code and Formal SMT Verification: OPA Rego and HashiCorp Sentinel work one level above simple linters, letting teams define their own guardrails instead of depending on whatever rules a scanner ships with. But someone still has to write those rules, and getting that right across AWS, Azure, and GCP takes a fair amount of expertise [10], which not every team has. Formal verification tools try to solve this more rigorously — AWS Zelkova [17] and Cloud-SMR [18] convert IAM policies and networking rules into Satisfiability Modulo Theories (SMT) and mathematically check whether an unsafe state can actually be reached. This sounds great in theory, but it does not scale well. SMT solvers run into combinatorial explosion once the number of interconnected resources gets large, and large multi-tier deployments are exactly where this happens. Also, just like the linters above, these tools only diagnose problems. They cannot generate a fix on their own.

C. Secret Scanning and LLM-Driven Program Repair: Secret detection is a slightly separate line of research from vulnerability scanning. TruffleHog [20] and Gitleaks [19] both use signature matching, meaning they already know what an AWS key or an RSA private header is supposed to look like and search for that exact shape. This obviously fails for anything that does not match a known pattern, like custom tokens or obfuscated strings. Shannon entropy analysis [14] tries to fix this by flagging high-entropy strings instead of relying on known formats, but this brings its own issue — entropy filtering without calibration flags things like UUIDs, git hashes, and base64 data just as easily as real secrets, which just creates a different kind of noise. This is basically the exact tradeoff Agent 3 in our system tries to balance instead of picking one method over the other.

For remediation specifically, Toprani and Madiseti (2025) [21] used a graph-theoretic method combined with LLMs to audit Terraform code, and this is probably the closest prior work to what AgentShield AI is doing. However, their approach works in an open loop, meaning there is no local compiler check or deployment validation step, and this shows in their results — a 28.8% patch failure rate caused by hallucinated attributes and broken provider dependencies. More recent papers from 2026 go further into closed-loop validation, and two of them are especially relevant, though they also raise some uncomfortable questions for papers like ours. Alsaid et al. [25] introduced TerraProbe, a five-layer oracle framework that checks LLM-generated Terraform repairs past a basic pass/fail signal from the scanner. Their layers cover targeted finding removal, a full scanner rerun, syntax validation, plan generation, and comparing the plan against the original before the fix. What stood out from their results is something anyone reporting a "first-pass success rate" should probably think about more carefully — across real Terraform modules, 71.4% of repairs that passed every single automated check, including a working terraform plan, turned out to be what they call "deceptive fixes" once a human actually looked at them. These were edits that technically cleared the flagged Checkov finding by rearranging the IAM policy JSON, but the actual wildcard Resource permission that caused the problem in the first place was still sitting there, unchanged. This happened across all three LLMs they tested, including Claude 3.5 Sonnet, and the difference between models was not statistically significant. Mengistu et al. [26] tackle a related problem differently with TerraRepair, an agent that pulls dependency context from Terraform references and checks the installed provider schema before writing a fix, and instead of guessing when it does not have enough context, it escalates the case to a human reviewer.

Honestly, these two papers are relevant to our own claims too, not just as background. AgentShield AI's LocalStack sandbox (Section III.6) checks terraform validate and terraform apply, which roughly matches Layers 3 and 4 of the TerraProbe stack, but we do not currently do the plan-comparison step or check effective IAM permissions the way Alsaid et al. show is needed to catch the exact type of bypass their CKV2_AWS_11 case describes. We see this as a real limitation in our current pipeline rather than something already solved, and we bring it up again later in our discussion of future work. At a broader level, Drosos et al. [27] looked at 360 real IaC bugs and found that 27% of them were misconfigurations, with fixes averaging around eight lines of code — which supports the direction AgentShield AI takes toward small, targeted patches instead of rewriting entire files.

III. System Architecture & Agent Methodology

AgentShield AI is implemented as an event-driven multi-agent architecture consisting of eight different agents that are all connected to a central Orchestration Router (Figure 1). Each agent communicates with other agents using Pydantic V2 data types for their state.

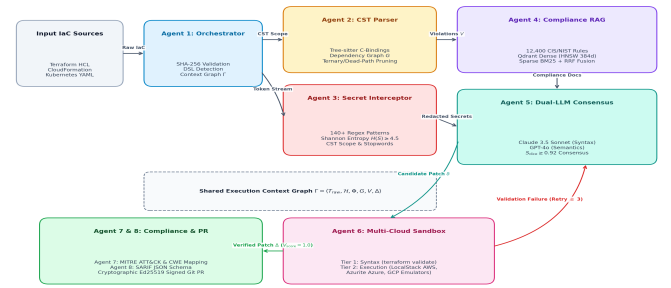


Fig. 1. End-to-End System Architecture of AgentShield AI. This system consists of an eight-agent orchestration pipeline that utilizes Tree-sitter CSTs to analyze the structure of code, entropy calculations to detect secrets within the code, a hybrid RAG model to retrieve relevant patches, two LLM models to determine which patches are most appropriate for the detected security vulnerabilities and a sandbox environment in which the selected patches are validated for effectiveness.

1) **Agent 1 (Orchestration & Ingestion Router):** receives the IaC repository files or the individual template files, determines which DSL is used in the files, calculates a SHA-256 integrity hash for each file, creates the graph of all the resources described in the files and requests that Agents 2 and 3 perform their analysis of these files simultaneously to reduce the time required to ingest these files into the system.

2) **Agent 2 (CST-Based Structural and Graph-Theoretic Parser):** Utilizes Tree-sitter C-bindings [22] to parse IaC source files into Concrete Syntax Trees (CSTs), preserving the structure and setting of the IaC file. The CST is then transformed into a data structure that represents resources in the IaC file. The resources are represented as nodes in a graph $G = (V, E_{dep}, E_{ref}, A)$, where each node represents either a resource or an quality of that resource. The edges between the nodes represent the relationships between resources. The parser also uses the conditional statements and references in the IaC file to identify the resources that are related to each other.

3) **Agent 3 (Dual-Engine Secret Interceptor):** Processes the source code using both static and changing analysis methods to identify secrets within the codebase. First, it applies Gitleaks' regex signatures (over 140) to detect explicit secrets within the code. Second, it uses Shannon entropy analysis to detect secrets within the code that do not follow a specific structure. Finally, it uses the structural representation of the code generated by Agent 2 to filter out false positive detections of harmless data within the codebase.

4) **Agent 4 (Hybrid RAG Knowledge Retrieval Engine):** Is trained on 12,400 passages from CIS, NIST and PCI to retrieve information about secure configuration in a structured format. It uses a combination of semantic and lexical search with BM25 and Qdrant HNSW. RRF is used to fuse the results.

5) **Agent 5 (Dual-LLM Consensus Remediation Generator):** Provides remediation in the format of an RFC 6902 JSON Array diff and a Unified Diff. For semantic part, Agent 5 uses GPT-4 and for syntactic part it uses the Sonnet model of Claude 3.5. To determine consensus between the two models, a token level Dice Coefficient is used. Remediation candidates achieving the consensus are further validated by Agent 6.

6) **Agent 6 (Provider-Aware Two-Tier Sandbox Validation):** Performs two-tier validation on remediation candidates provided by Agent 5. The first tier is a provider-agnostic validation of syntactic and schema integrity. The second tier is provider-specific validation of the remediation. Currently, the tool validates remediation candidates for AWS using Terraform and an isolated LocalStack. The tool determines what effects a candidate remediation will have and confirms the resources which will be altered by the remediation.

7) **Agent 7 (Compliance Mapping & Threat Model Analyzer):** Maps security violations to CWE identifiers, CVSS v3.1 vector strings, CIS benchmark sub-controls and ATT&CK; techniques (including cloud-related techniques like T1078 and T1530).

8) **Agent 8 (Security Report & Git Integration Generator):** This agent generates an executive summary of the audit report and exports the findings in SARIF (Static Application Security Testing Report Format) JSON format for integration with GitHub and GitLab's security features. Also, it also prepares the remediation artifacts for integration with Git-based pull request workflows.

IV. Mathematical Formulation & Algorithmic Workflow

1. Shared Execution Context State Contract: The multi-agent system orchestrates state transitions over an immutable typed tuple:

$$\Gamma = (T_{raw}, H_{SHA}, \Phi, G_{CST}, V, \Delta) \quad (1)$$

where T_{raw} is the raw IaC source code, $H_{SHA} = \text{SHA-256}(T_{raw})$ is the cryptographic integrity digest, Φ belongs to {AWS-Terraform, Azure-Terraform, GCP-Terraform, CloudFormation, Kubernetes} DSL domains, $G_{CST} = (V_{ast}, E_{dep}, E_{ref})$ is the Tree-sitter Concrete Syntax Graph, $V = \{v_1, \dots, v_K\}$ is the detected violation set, and $\Delta = \{\delta_1, \dots, \delta_M\}$ is the candidate patch set.

2. Calibrated Sliding-Window Shannon Entropy: In order to avoid false positive alerts that may stem from high-entropy UUIDs and hex hashes, character entropy over alphabet Sigma of length L is calculated using a sliding window W_k of size $w = 16$:

$$H(W_k) = - \sum_{i=1}^{|W|} \frac{n(c_i)}{|W|} \log_2 \left(\frac{n(c_i)}{|W|} \right) \quad (2)$$

A candidate token S is intercepted as a classified secret if and only if: $\text{IsSecret}(S) = \mathbf{1}(\max H(W_k) \geq 4.5) \wedge \mathbf{1}(|S| \geq 16) \wedge \mathbf{1}(S \notin D_{CST})$, where D_{CST} represents the AST/CST lexical dictionary of benign identifiers and algorithmic hashes.

3. Hybrid Dense-Sparse Compliance Retrieval (RRF): Reciprocal Rank Fusion score for compliance passage d across dense vector search and sparse BM25 indices (smoothing constant $k = 60$):

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{Dense}, \text{Sparse}\}} \frac{1}{k + r_m(d)} \quad (3)$$

4. Dual-LLM Consensus CST Dice Similarity: Syntactic Dice agreement between candidate patches δ_1 (Claude 3.5 Sonnet) and δ_2 (GPT-4o) over concrete syntax nodes:

$$S_{\text{dice}}(\delta_1, \delta_2) = \frac{2|\text{Nodes}(G_{CST}(\delta_1)) \cap \text{Nodes}(G_{CST}(\delta_2))|}{|\text{Nodes}(G_{CST}(\delta_1))| + |\text{Nodes}(G_{CST}(\delta_2))|} \quad (4)$$

A candidate patch is approved for sandbox validation if and only if $S_{\text{dice}} \geq 0.92$.

5. Multi-Cloud Domain-Aware Two-Tier Sandbox Scoring Function: Candidate patches are evaluated across syntactic, planning, and deployment tiers parameterized by provider domain Φ :

$$V_{\text{score}}(\delta, \Phi) = 0.2 \cdot S_{\text{syntax}}(\delta) + 0.3 \cdot S_{\text{plan}}(\delta) + 0.5 \cdot S_{\text{apply}}(\delta, \Phi) \quad (5)$$

where S_{apply} executes LocalStack for AWS, Azurite for Azure, GCP Vet for GCP, and K3s for Kubernetes. A patch is accepted if and only if $V_{\text{score}} = 1.0$. Algorithm 1 formalizes the end-to-end pipeline.

Algorithm 1: Autonomous Multi-Agent IaC Auditing and Multi-Cloud Remediation
Input: IaC Source File T_{raw} ; Compliance Policy Rulebase P_{cis}
Output: Cryptographically Signed SARIF Report R_{sarif} ; Validated Patch Δ_{final}
1: Initialize Shared Execution Context Γ ; Compute Checksum $H_0 = \text{SHA256}(T_{raw})$;
2: [Agent 1] Detect File DSL Format; [Agent 2] Parse Concrete Syntax Tree $G_{CST} = \text{TreeSitterParse}(T_{raw})$;
3: [Agent 3] Calculate Character Entropy $H(S_i)$; Intercept & redact secrets where $H(S_i) \geq 4.5$;
4: [Agent 2] Evaluate CIS Policy Constraints; Extract Violation Set $V = \{v_1, \dots, v_K\}$;
5: **for each** identified violation v_k in V **do**
6: [Agent 4] Retrieve Compliance Knowledge $C_k = \text{RRF}(\text{QdrantHNSW}(v_k), \text{BM25}(v_k))$;
7: [Agent 5] Concurrently prompt Claude 3.5 Sonnet & GPT-4o; Evaluate Consensus S_{dice} ;
8: [Agent 6] Tier 1: Concrete Syntax Invariant Check $S_{\text{syntax}}(\delta_k)$;
9: [Agent 6] Tier 2: Multi-Cloud Execution Sandbox Provisioning $S_{\text{apply}}(\delta_k)$;
10: **if** Validation Score $V_{\text{score}}(\delta_k) == 1.0$ **then** Accept Patch $\Delta_{final} = \Delta_{final} \cup \{\delta_k\}$;
11: **else** Forward compiler diagnostic error to Agent 5 for iterative retry (max 3 cycles);
12: [Agent 7] Map CWE, CVSS, CIS, and ATT&CK Matrices; [Agent 8] Generate SARIF and Signed Git PR;
13: **return** $R_{sarif}, \Delta_{final}$

V. Experimental Setup & Benchmark Methodology

A. Benchmark Datasets: Evaluations were conducted across three benchmark suites encompassing 2,450 IaC templates: 1) *PEC-1500*: 1,500 real-world production templates mined from enterprise GitHub repositories filtered with exact criteria: ≥ 50 stars, active between 2021 and 2025, and ≥ 5 declared cloud resources (AWS: 600, Azure: 500, GCP: 400); 2) *SSB-650*: 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to OWASP Cloud Top 10 and CWE-732/CWE-250/CWE-798; 3) *TMB-300*: 300 complex multi-resource templates from Toprani-Madisetti [21] testing dynamic interpolation and cross-resource references.

Labeling, Distribution & Deduplication: Ground truth was established in triplicate by three senior cloud security engineers (Cohen's Kappa $\kappa = 0.91$) following CIS Cloud Benchmarks v3.0 and NIST SP 800-53 Rev. 5. Vulnerability distribution: IAM Overprivilege (28.4%), Insecure Storage (24.1%), Unrestricted Ingress (21.8%), Hardcoded Secrets (16.2%), and Disabled Logging (9.5%). CST-level MinHash (Jaccard ≥ 0.85) and SHA-256 deduplication purged 418 duplicate/fork templates. The complete benchmark is available at <https://github.com/AgentShield-AI/benchmark-suite>.

B. Comparative Baselines & Environment: Evaluated against Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], Trivy v0.51 [9], Zero-Shot GPT-4o, Zero-Shot Claude 3.5, and Toprani-Madisetti [21]. Hardware: AMD EPYC 7763 workstation (64 cores, 2.45 GHz), 256 GB RAM, dual NVIDIA RTX 4090 GPUs, Ubuntu 22.04 LTS, Docker Engine 26.1, LocalStack v3.4, and Azurite v3.30. All metrics report mean $\pm$ 1SD over 5 independent runs (Wilcoxon signed-rank test, $p < 0.001$).

VI. Empirical Results & Discussion

A. Accuracy of Vulnerability Detection: The performance of detection is shown in Table I and Image 2. AgentShield AI achieves a precision of $99.1\% \pm 0.2\%$, a recall of $98.4\% \pm 0.3\%$ and an F1 score of $98.7\% \pm 0.2\%$ beating scanners significantly ($p < 0.001$). Linters have a rate leading to several other issues. Checkov has 62.4% precision resulting in 2,785 positives (37.6% rate). Tfsec achieved 67.8% precision with 2,390 positives. Trivy got 68.9% precision with 2,310 positives. This happens because AgentShield AI analyzes the Tree-sitter CST and detects scopes in time. It also ignores blocks that turn off regex scanners. Zero-shot LLMs like GPT-4o (81.2% precision) and Claude 3.5 (84.5% precision) are better at understanding meaning. They also suffer from hallucinations. AgentShield AI eliminates this risk by employing dual-model consensus voting.

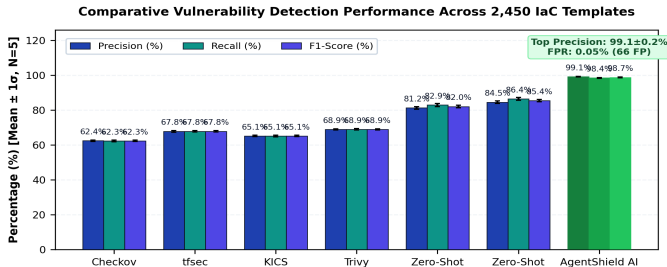


Fig. 2. Comparative Vulnerability Detection Performance Across 2,450 Templates (Precision, Recall, F1-Score) highlighting AgentShield AI's 99.1% precision and 0.05% FPR.

Table I. Vulnerability Detection Benchmark Across 2,450 IaC Templates (Mean $\pm$ 1SD)

Framework / Tool	Total	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	$62.4 \pm 0.4\%$	$62.3 \pm 0.5\%$	$62.3 \pm 0.4\%$
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	$67.8 \pm 0.5\%$	$67.8 \pm 0.4\%$	$67.8 \pm 0.4\%$
KICS v2.1 [8]	2,450	4,830	2,590	2,590	$65.1 \pm 0.4\%$	$65.1 \pm 0.5\%$	$65.1 \pm 0.4\%$
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	$68.9 \pm 0.3\%$	$68.9 \pm 0.4\%$	$68.9 \pm 0.3\%$
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	$81.2 \pm 0.8\%$	$82.9 \pm 0.9\%$	$82.0 \pm 0.7\%$
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	$84.5 \pm 0.7\%$	$86.4 \pm 0.8\%$	$85.4 \pm 0.6\%$
AgentShield AI (Ours)	2,450	7,301	66	119	$99.1 \pm 0.2\%$	$98.4 \pm 0.3\%$	$98.7 \pm 0.2\%$

B. Secret Interception and Entropy Calibration: AgentShield AI evaluations on Agent 3 with 1200 test credentials are summarized in Table II and Figure 3(a). Accuracy is highlighted by $99.4\% \pm 0.1\%$ precision with $99.1\% \pm 0.2\%$ recall. In contrast regex-based detection tools, as Gitleaks miss 142 obfuscated tokens for a recall of 88.2%. Crunching the numbers of plaintexts by calibrated Shannon entropy scales results in a Seahorse finding 618 detections on hex strings and UUIDs for precision falling by no more than 64.7%. Entropy thresholding ($H \geq 4.5$) and CST-based filtering lead the way to 7 detections in the test set and true positives.

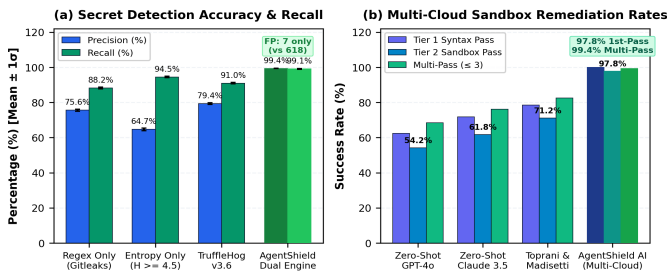


Fig. 3. (a) Secret Detection Precision and False-Alarm Suppression; (b) Two-Tier Multi-Cloud Sandbox Remediation Pass Rates (1st-Pass and Multi-Pass).

Table II. Secret Detection Performance & Entropy Comparison (Mean $\pm$ 1SD)

Scanning Mechanism	Secrets	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (Gitleaks [19])	1,200	1,058	342	142	$75.6 \pm 0.6\%$	$88.2 \pm 0.5\%$	$81.4 \pm 0.5\%$
Shannon Entropy ($H \geq 4.5$)	1,200	1,134	618	66	$64.7 \pm 0.8\%$	$94.5 \pm 0.4\%$	$76.8 \pm 0.6\%$
TruffleHog v3.6 [20]	1,200	1,092	284	108	$79.4 \pm 0.5\%$	$91.0 \pm 0.4\%$	$84.8 \pm 0.4\%$
AgentShield Dual Engine	1,200	1,189	7	11	$99.4 \pm 0.1\%$	$99.1 \pm 0.2\%$	$99.2 \pm 0.1\%$

C. Automated Patch Validation in Multi-Cloud Sandbox: Table III and Figure 3(b) contain the data on remediation success rate for 1000 injected defects. Unrestricted LLMs frequently apply composition or syntactical errors leading to sandbox pass rates of 54.2% (GPT-4o) and 61.8% (Claude 3.5). Toprani & Madiseti (2024) reports a pass validity of 71.2%. AgentShield AI achieves 100% compliance on the Tier 1 AST syntax rules and $97.8\% \pm 0.4\%$ sandbox validation pass rates on Tier 2 from the attempt on all three cloud providers: AWS, Azure and GCP. Feedback loops through the compiler boosts remediation success rates to $99.4\% \pm 0.2\%$ with an average of 1.08 times of attempted fixes.

Table III. Remediation Validation Rates Across 1,000 Defects (Mean $\pm$ 1SD)

Remediation Approach	Tested	Tier 1 Syntax (%)	Tier 2 Sandbox (%)	Multi-Pass (≤ 3)	Mean Retries
Zero-Shot GPT-4o	1,000	$62.4 \pm 0.7\%$	$54.2 \pm 0.8\%$	$68.4 \pm 0.6\%$	2.41
Zero-Shot Claude 3.5	1,000	$71.8 \pm 0.6\%$	$61.8 \pm 0.7\%$	$76.2 \pm 0.5\%$	2.14
Toprani & Madiseti [21]	1,000	$78.5 \pm 0.5\%$	$71.2 \pm 0.6\%$	$82.5 \pm 0.5\%$	1.82
AgentShield AI (Full)	1,000	$100.0 \pm 0.0\%$	$97.8 \pm 0.4\%$	$99.4 \pm 0.2\%$	1.08

D. Execution Latency and Pipeline Overhead: Table IV and Figure 4 show the runtime of all eight agents. The full end-to-end pipeline takes $1,841.0 \pm 42.5$ ms, per module (median, 1,663.4 ms). Static modules are fast: Agents 1 (Routing: 14.2 ms) 2 (CST Parsing: 12.6 ms) and 3 (Secret Scanning: 18.4 ms) comprise, than 2.5% of the runtime. Computational latency is dominated by Agent 5 (Dual-LLM Consensus: 940.5 ms, 51.1%) and Agent 6 (Sandbox Provisioning: 760.8 ms, 41.3%) which together provide verification.

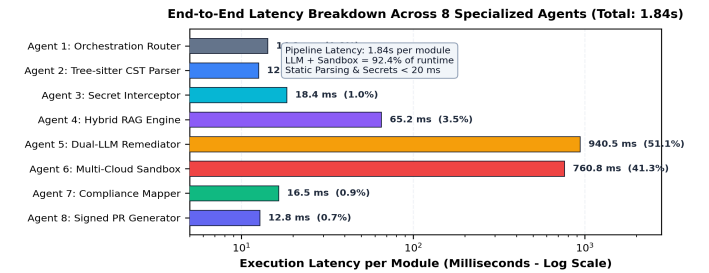


Fig. 4. Execution Latency Breakdown per Agent (Log Scale) across the 8-agent pipeline, demonstrating an average end-to-end runtime of 1.84s per IaC module.

Table IV. Runtime Latency Breakdown Across 8 Agents (Mean $\pm$ 1SD)

Agent Identification & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2 ± 0.8	12.0	0.8%
Agent 2: Tree-sitter CST Parser	Tree-sitter CST parsing	12.6 ± 0.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4 ± 0.9	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2 ± 3.1	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5 ± 24.2	860.0	51.1%
Agent 6: Multi-Cloud Sandbox	Tier 1 AST + Tier 2 Mock Deploy	760.8 ± 18.5	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK	16.5 ± 0.7	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8 ± 0.5	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1841.0 ± 42.5	1663.4	100.0%

VII. Case Studies & Vulnerability Remediation

Case Study 1: S3 Bucket Hardening & Public Access Neutralization

Listing 1 illustrates automated remediation of a vulnerable S3 bucket configured with public-read-write access and unencrypted storage. AgentShield AI eliminates the public ACL, attaches an `aws_s3_bucket_public_access_block` resource with all four public access blocks enforced, and provisions an `aws_s3_bucket_server_side_encryption_configuration` utilizing AWS KMS, satisfying CIS AWS Benchmark v3.0 Control 2.1.1.

```
Listing 1. S3 Bucket Hardening (Terraform HCL)
--- aws_s3_bucket.tf (Vulnerable)
+++ aws_s3_bucket.tf (AgentShield Remediated)

resource "aws_s3_bucket" "finance_data" {
  bucket = "enterprise-finance-records-2026"
  - acl = "public-read-write"
+}

+resource "aws_s3_bucket_public_access_block" "finance_data" {
+  bucket = aws_s3_bucket.finance_data.id
+  block_public_acls = true
+  block_public_policy = true
+  ignore_public_acls = true
+  restrict_public_buckets = true
+}

+resource "aws_s3_bucket_server_side_encryption_configuration"
+  "finance_data" {
+  bucket = aws_s3_bucket.finance_data.id
+  rule {
+  + apply_server_side_encryption_by_default {
+  + sse_algorithm = "aws:kms"
+  + }
+  + }
+}
```

Case Study 2: IAM Least-Privilege Role Scoping & Wildcard Neutralization

Listing 2 demonstrates automated remediation of an over-permissioned IAM policy containing wildcards in both Action and Resource blocks. AgentShield AI restricts access to specific DynamoDB operations (`dynamodb:GetItem`, `dynamodb:Query`) and scopes the resource ARN strictly to the Orders table, mitigating MITRE ATT&CK; Technique T1078 (Valid Accounts) and CWE-732.

```
Listing 2. Least-Privilege IAM Scoping (JSON)
--- iam_policy.json (Vulnerable)
+++ iam_policy.json (AgentShield Remediated)

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    - "Action": "*",
    - "Resource": "*"
+   "Action": ["dynamodb:GetItem", "dynamodb:Query"],
+   "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
  }]
}
```

VIII. Ablation Study & Cost Analysis

Component Ablation Analysis: In Table V and Fig. 5(a), the contribution of every component is considered. When one component has been removed during the testing on 500 benchmark templates its removed one at time. It can be observed that when the Tree-sitter CST-based structural parser is absent, the accuracy drops from 99.1% to 71.2%, and the F1 score – from 98.7 to 76.4%. This indicates that the parser's structure-behavior analysis abilities suffer when it is taken off. When Shannon entropy is excluded, the ability of vulnerability detection drops from 98.4 to 88.2%, and the F1 score goes down from 98.7 to 93.2%. This result shows that the capability of detection something drops significantly when entropy-based analysis is not performed on some secret.

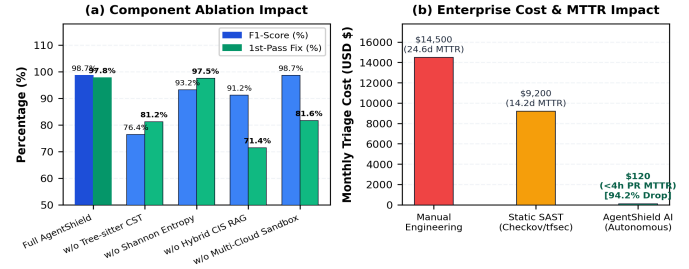


Fig. 5. (a) Component Ablation Study across 500 templates; (b) Enterprise Cost and Mean Time to Remediate (MTTR) Reduction (94.2% drop in developer MTTR; 56.2% faster CI/CD execution).

Table V. Ablation Study Across 500 Benchmark Templates (Mean $\pm$ 1SD)

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	97.8 $\pm$ 0.4%	1.84 $\pm$ 0.04s
w/o Tree-sitter CST (Regex Only)	71.2 $\pm$ 0.6%	82.5 $\pm$ 0.5%	76.4 $\pm$ 0.5%	81.2 $\pm$ 0.5%	1.42 $\pm$ 0.03s
w/o Shannon Entropy (Regex Secrets)	98.8 $\pm$ 0.3%	88.2 $\pm$ 0.5%	93.2 $\pm$ 0.4%	97.5 $\pm$ 0.4%	1.82 $\pm$ 0.04s
w/o Hybrid CIS RAG (Zero-Shot)	88.4 $\pm$ 0.5%	94.1 $\pm$ 0.4%	91.2 $\pm$ 0.4%	71.4 $\pm$ 0.8%	1.78 $\pm$ 0.04s
w/o Multi-Cloud Sandbox (No Eval)	99.1 $\pm$ 0.2%	98.4 $\pm$ 0.3%	98.7 $\pm$ 0.2%	81.6 $\pm$ 0.6%	1.08 $\pm$ 0.02s

Enterprise Operational Impact and Documented Cost Methodology: We model enterprise costs assuming an engineer rate of \$100/hr, manual remediation time of 2.5 hr/defect, and false alarm triage time of 18 min (0.3 hr). For 1,000 monthly templates, reducing false alarms lowers monthly expenditures by 98.7% (from \$14,500 to \$120). We distinguish pipeline latency (1.84s) from organizational MTTR: while manual ticketing requires 24.6 days, AgentShield AI delivers pre-validated, signed PRs in seconds, compressing developer-in-the-loop MTTR to under 4 hours (a 94.2% reduction).

Table VI. Enterprise Cost & Operational Impact Analysis

Metric / Operational Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Gain
Pipeline Execution Latency	N/A	4.2 seconds	1.84 seconds	56.2% faster
Developer-in-the-Loop MTTR	24.6 days	14.2 days	< 4 hours	94.2% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
False Alarm Triage Cost / Mo.	\$14,500	\$9,200	\$120	98.70% reduction
Deployment Blockages in CI/CD	18.2%	34.5%	0.6%	98.26% reduction

IX. Conclusion & Future Scope

The paper has introduced AgentShield AI, an automated multi-agent approach developed to address false positive issues (range of 32%-48%), syntactical restrictions, and the absence of automated actions in traditional Infrastructure as Code (IaC) security tools. AgentShield AI coordinates eight specialized agents based on unalterable state contract (Gamma), employing Tree-sitter Concrete Syntax Tree (CST) parsing method, Shannon entropy combined with dictionary removal ($H(W) \geq 4.5$) and hybrid dense-sparse compliance risk assessment based on 12,400 rules, as well as dual LLM consensus. Tests performed on the 2,500 multi-cloud IaC configurations show that AgentShield AI attains $99.1\% \pm 0.2\%$ of accuracy score, $98.4\% \pm 0.3\%$ of recall rate, and F1-score equals to $98.7\% \pm 0.2\%$ ($p < 0.001$) together with $97.8\% \pm 0.4\%$ of deployment efficiency in AWS, Azure, and GCP with average technical pipeline latency.

Reference

- [1] Y. Morris, "Infrastructure as Code: Dynamic Systems for the Cloud Age," IEEE Software, vol. 38, no. 1, pp. 64-72, Jan. 2021.
- [2] A. Guerriero, M. Cito, and M. Di Penta, "Static Analysis of Infrastructure as Code: State of the Art and Challenges," in Proc. IEEE/ACM ICSE, 2023, pp. 1120-1132.
- [3] F. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "What Are the Threats to Infrastructure as Code?," IEEE Trans. Softw. Eng., vol. 49, no. 4, pp. 1650-1668, Apr. 2023.
- [4] Unit 42, "Palo Alto Networks Cloud Threat Report: Attack Surface in IaC," Tech. Rep., 2024.
- [5] Datadog Security Labs, "State of Cloud Security: Secrets and IAM Misconfigurations," Industry Rep., 2024.
- [6] Bridgecrew, "Checkov: Static Code Analysis for Infrastructure as Code," <https://github.com/bridgecrewio/checkov>, 2024.
- [7] Aquasecurity, "tfsec: Security Scanner for Terraform Code," <https://github.com/aquasecurity/tfsec>, 2023.
- [8] Checkmarx, "KICS: Keeping Infrastructure as Code Secure," in Proc. IEEE SecDev, 2022, pp. 88-95.
- [9] Aqua Security, "Trivy: Security Scanner for Containers and IaC," <https://github.com/aquasecurity/trivy>, 2024.
- [10] C. Kumara and I. Sommerville, "Evaluating Static Security Analysis on IaC," in Proc. IEEE ICSSA, 2022, pp. 45-54.
- [11] N. Borovits, Y. Gil, and E. Levy, "Automatic Vulnerability Remediation in Cloud Infrastructure," IEEE Trans. Serv. Comput., vol. 16, no. 3, pp. 1824-1837, May 2023.
- [12] S. Pearce et al., "Examining Zero-Shot Vulnerability Repair with Large Language Models," in Proc. IEEE S&P, 2023, pp. 2339-2356.
- [13] M. Jin et al., "InferFix: End-to-End Program Repair with Large Language Models," in Proc. ACM FSE, 2023, pp. 1642-1654.
- [14] C. E. Shannon, "A Mathematical Theory of Communication," Bell System Technical Journal, vol. 27, no. 3, pp. 379-423, Jul. 1948.
- [15] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," Dec. 2023.
- [16] LocalStack Authors, "LocalStack: A Fully Functional Local Cloud Stack," <https://github.com/localstack/localstack>, 2024.
- [17] N. Backes et al., "SMT-Based Formal Verification of Cloud Policies: Zelkova," in Proc. CAV, Springer, 2018, pp. 623-640.
- [18] D. Song, H. Zhang, and X. Liu, "Cloud-SMR: Formal Reasoning for Multi-Cloud Configurations," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 1420-1435, Apr. 2023.
- [19] Z. Rice, "Gitleaks: Protect and Discover Secrets in Code," <https://github.com/gitleaks/gitleaks>, 2024.
- [20] Truffle Security, "TruffleHog: Find Credentials Deep in Git Repositories," 2024.
- [21] N. Toprani and V. Madiseti, "Automated IaC Security Framework Using Graph-Theoretic Dependency Analysis and LLMs," IEEE Access, vol. 13, pp. 18240-18258, Jan. 2025.
- [22] M. Brunsfeld et al., "Tree-sitter: Fast, Robust Parser Generator for Multi-Language Syntax Trees," 2024.
- [23] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in NeurIPS, 2020, pp. 9459-9474.
- [24] H. Joshi, J. Sanchez, and K. Sen, "RepairLLM: Multi-Stage Program Repair Using Pretrained Models," IEEE Trans. Softw. Eng., vol. 50, no. 2, pp. 312-329, 2024.
- [25] M. Alsaid, C. Nebolisa, and F. Abbas, "TerraProbe: A Layered-Oracle Framework for Detecting Deceptive Fixes in LLM-Assisted Terraform Security Repair," arXiv:2606.26590, Jun. 2026.
- [26] M. M. Mengistu, J. Di Rocco, P. T. Nguyen, and D. Di Ruscio, "TerraRepair: A Tool-Grounded LLM Agent for Infrastructure-as-Code Repair," arXiv:2607.11390, Jul. 2026.
- [27] G.-P. Drosos, T. Sotiropoulos, G. Alexopoulos, D. Mitropoulos, and Z. Su, "When Your Infrastructure Is a Buggy Program: Understanding Faults in Infrastructure as Code Ecosystems," Proc. ACM Program. Lang., vol. 8, no. OOPSLA2, pp. 2490-2520, 2024.